FinanzasESP

Plataforma de finanzas personales con ESP32 y tickets de compra

Entregable v01 — Modelo User · Hasheo · Token · MinIO

Documentación, explicación y justificación de tecnologías,
modelos de base de datos y modelos de inteligencia artificial

Owen Morales
Desarrollo Backend con FastAPI
Repositorio GitHub · rama **v01**
26/10/2026

Contenido

- 1. Introducción
- 2. Desarrollo
- 2.1 Arquitectura general
- 2.2 Tecnologías y justificación
- 2.3 Modelos de base de datos
- 2.4 Modelo User
- 2.5 Hasheo de contraseñas
- 2.6 Tokens (JWT)
- 2.7 Almacenamiento con MinIO
- 2.8 Integración con el ESP32
- 2.9 Modelos de inteligencia artificial
- 2.10 Entorno, pruebas y evidencias
- 2.11 Repositorio y rama v01
- 3. Conclusión
- Referencias

1. Introducción

La mayoría de las personas no sabe con precisión en qué gasta su dinero. Los tickets de compra contienen exactamente esa información —tienda, fecha, productos, cantidades y precios— pero terminan en una bolsa o en la basura. Capturarlos a mano en una hoja de cálculo es tedioso y, por eso, casi nadie lo hace de forma constante.

FinanzasESP es una plataforma que resuelve ese problema: el usuario sube la foto o el PDF de su ticket desde un panel web o desde un dispositivo **ESP32** (con el firmware de agente ESP-Claw), la imagen se guarda de forma segura en **MinIO** y, en la siguiente versión, un modelo de inteligencia artificial extrae los productos y precios para construir el historial de gastos, compararlo contra un presupuesto mensual y detectar cambios de precio entre compras.

Este documento corresponde al **primer entregable (rama v01)**, cuyo alcance es la base de seguridad e infraestructura sobre la que se construye todo lo demás:

- **Modelo User**: la entidad dueña de los tickets, dispositivos y preferencias financieras.
- **Hasheo**: almacenamiento seguro de contraseñas con Argon2id.
- **Token**: autenticación con JWT (access, refresh con rotación y token de dispositivo).
- **MinIO**: almacenamiento de objetos para avatares y tickets de compra.

Además del mínimo pedido, se desarrollaron: vinculación de dispositivos ESP32 con llaves propias, subida de tickets (multipart y base64), un panel web, migraciones con Alembic, 22 pruebas automatizadas y la skill para el agente ESP-Claw. También se documenta el esquema completo de base de datos y los modelos de IA que se integrarán en la v02.

2. Desarrollo

2.1 Arquitectura general

El sistema sigue una arquitectura cliente-servidor en capas. Los clientes (navegador y ESP32) solo hablan con la API; la API es la única que accede a PostgreSQL y a MinIO. Los archivos nunca pasan por la base de datos: en PostgreSQL solo se guarda la *llave* del objeto en MinIO.

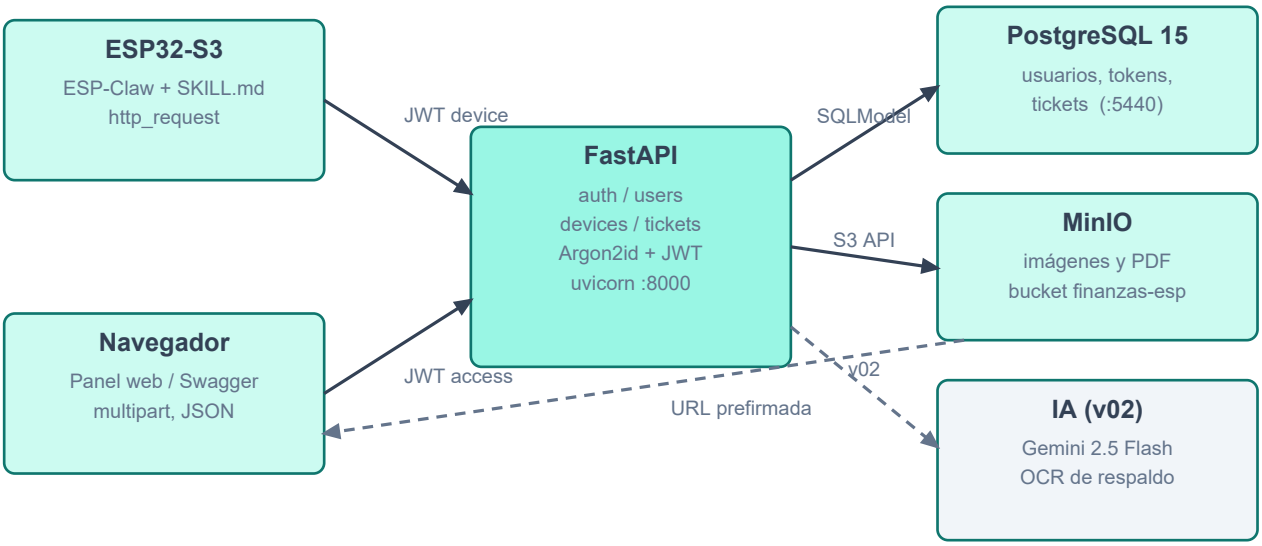


Figura 1. Arquitectura de FinanzasESP (líneas punteadas: componentes de la v02).

El código respeta la estructura en capas usada a lo largo del curso:

Carpeta	Responsabilidad
core/	Configuración desde `.env` (pydantic-settings) y seguridad: Argon2id, JWT, llaves de dispositivo.
db/	Motor de SQLModel y la dependencia `get_session`.
models/	Tablas (SQLModel, table=True) y enumeraciones.
schemas/	Contratos de entrada y salida (Pydantic). Nunca exponen el hash.
services/	`storage_service`: encapsula MinIO; el resto de la API no conoce S3.
api/v1/	Routers auth, users, devices, tickets y dependencias de autenticación.
alembic/	Migraciones versionadas del esquema.
tests/	Pruebas con SQLite en memoria y MinIO simulado.

2.2 Tecnologías y justificación

Tecnología	Rol	Justificación	Alternativa descartada
FastAPI	Framework web	Validación automática con Pydantic, documentación OpenAPI/Swagger generada sola, inyección de dependencias (ideal para auth y sesiones) y alto rendimiento.	Flask (sin validación ni docs integradas), Django (demasiado pesado para una API).
SQLModel	ORM	Una clase sirve como tabla y como modelo Pydantic; menos código duplicado. Basado en SQLAlchemy 2, así que soporta relaciones, cascadas y Alembic.	SQLAlchemy puro (más verboso).
PostgreSQL 15	Base de datos	ACID, tipo NUMERIC exacto para dinero (no float), UUID y ENUM nativos, índices y restricciones UNIQUE compuestas. Corre en Docker.	SQLite (sin concurrencia real), MongoDB (los datos financieros son relacionales).
Alembic	Migraciones	Cambios del esquema versionados y reversibles; la migración inicial también siembra las categorías.	create_all (no versiona ni migra).
Argon2id (passlib + argon2-cffi)	Hasheo	Ganador de la Password Hashing Competition (2015); OWASP lo recomienda en primer lugar. Resistente a GPU por consumo de memoria.	bcrypt (trunca a 72 bytes, no es memory-hard), SHA-256 (demasiado rápido).
JWT (python-jose)	Tokens	Estándar RFC 7519; el access token se valida sin consultar sesiones, y funciona igual para navegador y ESP32 con el header Authorization.	Sesiones con cookie (incómodas para IoT).
MinIO + SDK minio	Archivos	Almacenamiento de objetos compatible con S3, gratuito y local. Con el mismo código se podría migrar a AWS S3. URLs prefirmadas.	Guardar archivos en la BD (la infla) o en disco (no escala).
Docker Compose	Infraestructura	PostgreSQL y MinIO idénticos en cualquier PC con `docker compose up -d`; puertos propios (5440, 9030, 9031).	Instalación manual de servicios.
ESP-Claw (ESP32-S3)	Dispositivo	Firmware de agente de Espressif: se programa con un SKILL.md en lenguaje natural y la herramienta http_request, sin recompilar firmware.	Sketch Arduino (cada cambio requiere flashear).
pytest	Pruebas	Pruebas rápidas con TestClient; las dependencias de BD y MinIO se sustituyen con dependency_overrides.	Pruebas manuales en Swagger.

2.3 Modelos de base de datos

El esquema completo se creó desde la v01 para que la base de datos ya refleje el dominio del proyecto, aunque `ticket_items` y `categories` se llenarán con la IA en la v02. Todas las llaves primarias son **UUID**: no se pueden enumerar recursos probando /users/1, /users/2, y el ESP32 puede generar referencias sin chocar.

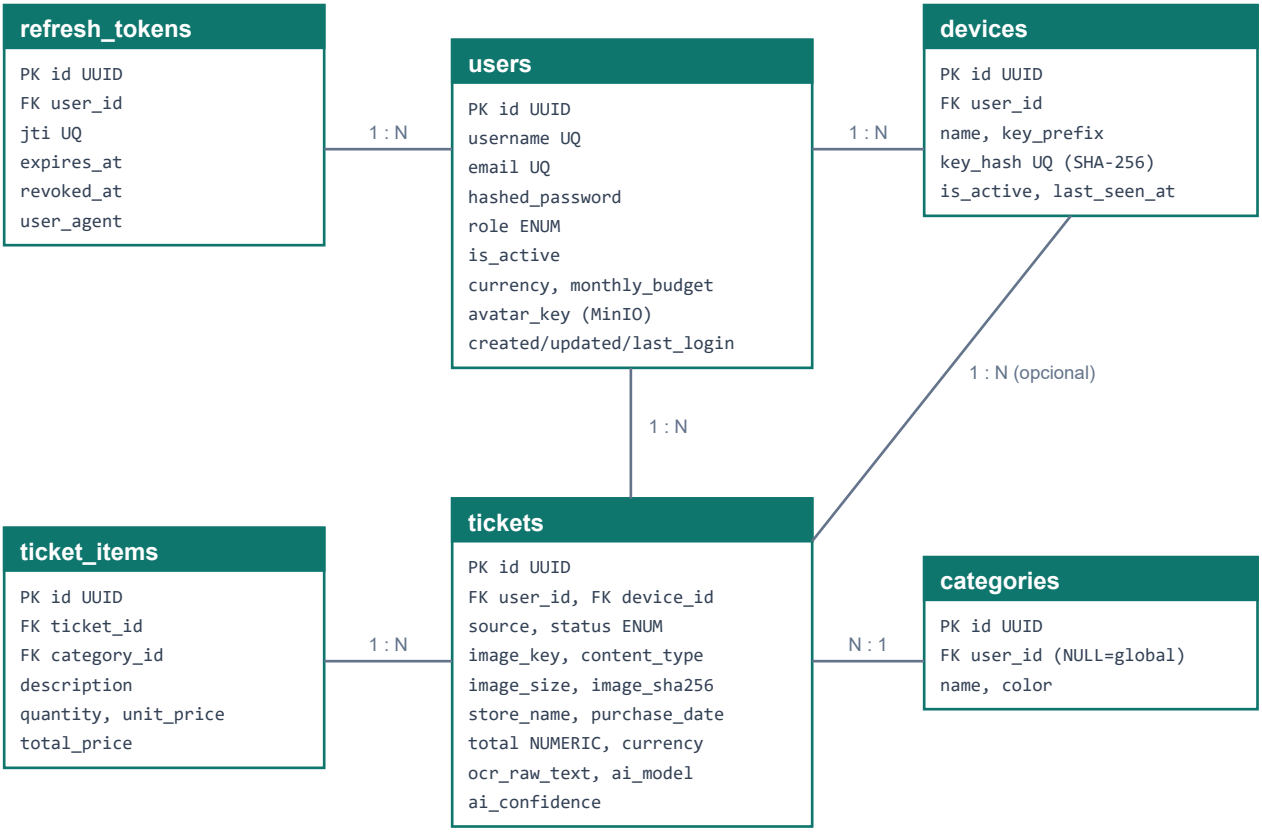


Figura 2. Diagrama entidad-relación (6 tablas).

Tabla	Propósito	Decisiones de diseño
users	Cuenta del usuario.	username y email UNIQUE + índice; dinero en NUMERIC(12,2).
refresh_tokens	Sesiones abiertas.	Solo se guarda el jti (id del token), nunca el token. revoked_at permite cerrar sesión y detectar reutilización.
devices	ESP32 vinculados.	Se guarda el SHA-256 de la llave (UNIQUE, indexado). Revocar = is_active=false, para conservar el origen de sus tickets.
tickets	Ticket subido.	UNIQUE(user_id, image_sha256) evita duplicados. device_id ON DELETE SET NULL. Campos de IA para trazabilidad.
ticket_items	Productos del ticket.	ON DELETE CASCADE con el ticket; quantity con 3 decimales (kilos).
categories	Clasificación del gasto.	user_id NULL = categoría global; la migración siembra 8 (Despensa, Restaurantes, Transporte, Salud, Hogar, Entretenimiento, Servicios, Otros).

Las relaciones usan `ondelete="CASCADE"` en la llave foránea y `cascade_delete=True` en SQLModel: al borrar un usuario se eliminan sus tokens, dispositivos, tickets y productos en una sola transacción;

después la API borra todo su prefijo en MinIO.

2.4 Modelo User

El usuario es la raíz del dominio: todo lo demás le pertenece. El modelo combina datos de identidad, seguridad y preferencias financieras.

```
class User(SQLModel, table=True):
    __tablename__ = "users"

    id: uuid.UUID = Field(default_factory=uuid.uuid4, primary_key=True)
    username: str = Field(index=True, unique=True, min_length=3, max_length=50)
    email: str = Field(index=True, unique=True, max_length=320)
    full_name: str | None = Field(default=None, max_length=120)
    hashed_password: str = Field(max_length=255)      # solo el hash Argon2id
    role: UserRole = Field(default=UserRole.user)      # user | admin
    is_active: bool = Field(default=True)
    currency: str = Field(default="MXN", min_length=3, max_length=3)
    monthly_budget: Decimal | None = Field(default=None, max_digits=12, decimal_places=2)
    avatar_key: str | None = Field(default=None, max_length=255)   # llave en MinIO
    created_at / updated_at / last_login_at: datetime

    refresh_tokens / devices / tickets: Relationship(cascade_delete=True)
```

Campo	Justificación
id UUID	No enumerable; seguro de exponer en URLs y en el ESP32.
username / email	Únicos e indexados; se normalizan a minúsculas para que ANA y ana sean la misma cuenta. El login acepta cualquiera de los dos.
hashed_password	Nunca se guarda ni se devuelve la contraseña; el esquema de respuesta no incluye el hash.
role	Autorización: solo admin puede listar usuarios (GET /api/v1/users/).
is_active	Permite suspender una cuenta sin borrar su historial; sus tokens dejan de servir (403).
currency, monthly_budget	Base de la v03: comparar gastos del mes contra el presupuesto.
avatar_key	Se guarda la llave del objeto, no la URL, porque las URLs prefirmadas caducan.
last_login_at	Auditoría básica de actividad.

Los esquemas separan lo que entra de lo que sale: `UserCreate` valida formato de usuario (letras, números, punto, guion), correo con `EmailStr` y la política de contraseña; `UserUpdate` solo permite cambiar nombre, correo, moneda y presupuesto (no el rol ni is_active); `UserResponse` expone un campo calculado `has_avatar` en lugar de la llave interna.

2.5 Hasheo de contraseñas

Una contraseña nunca debe guardarse en texto plano ni cifrada (lo cifrado se puede descifrar). Se guarda un **hash**: una función de un solo sentido. Para contraseñas no sirve cualquier hash: SHA-256 calcula miles de millones de hashes por segundo en una GPU, así que una contraseña filtrada se rompe rápido. Se necesita un algoritmo **lento a propósito y con sal**.

Se eligió **Argon2id** con los parámetros mínimos que recomienda OWASP:

Parámetro	Valor	Significado
Variante	Argon2id	Híbrido: resiste ataques de canal lateral (Argon2i) y de GPU (Argon2d).
memory_cost (m)	19 456 KiB (19 MiB)	Cada intento obliga a reservar 19 MiB: una GPU con miles de núcleos no tiene memoria para probar tantas contraseñas en paralelo.
time_cost (t)	2	Número de pasadas sobre la memoria.
parallelism (p)	1	Hilos por hash.
Sal	16 bytes aleatorios	Distinta en cada hash: dos usuarios con la misma contraseña tienen hashes diferentes y las tablas arcoíris no sirven.

El hash resultante es autodescriptivo: incluye algoritmo, versión, parámetros y sal. Así se ve en la base de datos real:

```
$argon2id$v=19$m=19456,t=2,p=1$Z2wNoTTGmHOOce59z/mfsw$i2z2fU...
 algoritmo  ver  parámetros      sal (base64)         hash
```

Mejoras implementadas sobre el hasheo básico:

- **Rehash transparente**: el login usa `verify_and_update`; si en el futuro se suben los parámetros en el .env, el hash de cada usuario se actualiza la próxima vez que inicia sesión.
- **Protección contra timing attacks**: si el usuario no existe se verifica contra un hash señuelo, para que la respuesta tarde lo mismo y no revele qué cuentas existen. El mensaje de error también es el mismo.
- **Política de contraseña**: 8 a 128 caracteres con al menos una letra y un número. A diferencia de bcrypt, Argon2 no trunca a 72 bytes, por lo que se permiten frases largas.
- **Llaves de dispositivo con SHA-256**: las llaves del ESP32 son aleatorias de 256 bits (`secrets.token_urlsafe(32)`); no se pueden adivinar, así que basta un hash rápido, que además permite buscarlas por índice. Argon2 se reserva para secretos elegidos por humanos.

```
pwd_context = CryptContext(
    schemes=["argon2"], deprecated="auto", argon2__type="ID",
    argon2__time_cost=settings.ARGON2_TIME_COST,       # 2
    argon2__memory_cost=settings.ARGON2_MEMORY_COST,   # 19456 KiB
    argon2__parallelism=settings.ARGON2_PARALLELISM,   # 1
)
_DUMMY_HASH = pwd_context.hash("usuario-inexistente")   # contra timing attacks
```

2.6 Tokens (JWT)

Después del login, el cliente no vuelve a enviar su contraseña: envía un **JSON Web Token** en el header `Authorization: Bearer <token>`. El token está firmado con HMAC-SHA256 y una `SECRET_KEY` de 512 bits generada con `secrets`; si alguien modifica un solo carácter, la firma deja de coincidir y la API responde 401.

Tipo	Duración	Obtención	Uso
access	30 min	POST /auth/token (OAuth2 form)	Todas las rutas del usuario.
refresh	7 días	Junto con el access	Solo POST /auth/refresh y /auth/logout. Se rota en cada uso.
device	60 min	POST /auth/device-token con la llave del ESP32	Subir y consultar tickets. Prohibido: borrar, cambiar contraseña, crear dispositivos (403).

Claims de cada token:

```
{
  "sub":  "9342f8b7-3eda-4b8c-938b-1e22b4c91022",   # id del usuario
  "type": "access",                                 # access | refresh | device
  "jti":  "99264497b6bc4e2c...",                    # id único del token
  "iss":  "finanzas-esp",                           # emisor (se valida)
  "iat":  1793068631, "exp": 1793070431,            # emitido / expira
  "role": "user"                                    # solo access; device lleva "dev"
}
```

El claim `type` evita la confusión de tokens: un refresh token presentado como Bearer es rechazado, y uno de dispositivo no sirve en rutas exclusivas del usuario.

Rotación de refresh tokens y detección de robo

El access token es corto para limitar el daño si se filtra. El refresh token dura más, por lo que se protege con **rotación**: cada vez que se usa se revoca y se emite uno nuevo. Si alguien presenta un refresh ya revocado significa que hay dos copias del token (la del usuario y la de un atacante), así que la API revoca **todas** las sesiones del usuario.

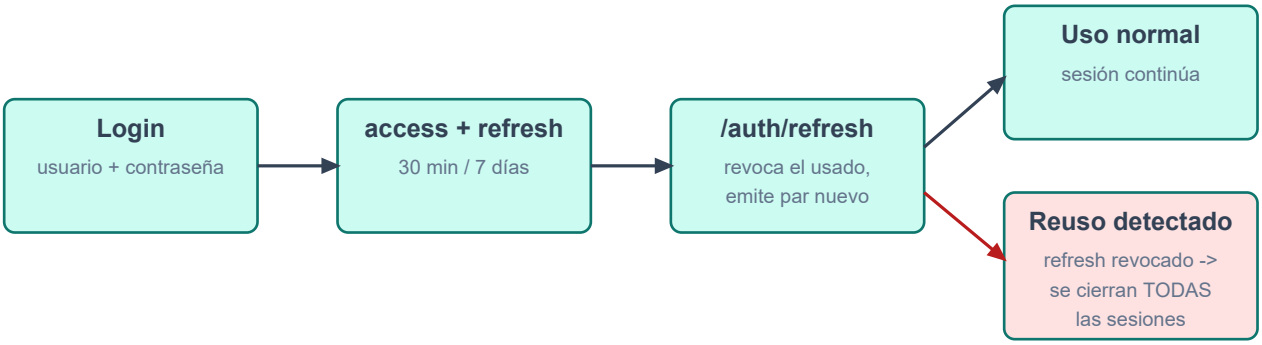


Figura 3. Rotación de refresh tokens con detección de reutilización.

- **Logout** revoca el refresh token (idempotente).
- **Cambiar la contraseña** revoca todos los refresh tokens: se cierran las sesiones en otros equipos.

- **Revocar un ESP32** invalida su token de inmediato: la dependencia comprueba `device.is_active` en cada petición.
- Usuario inactivo: 403 aunque el token tenga firma válida.

2.7 Almacenamiento con MinIO

Los archivos binarios (fotos y PDF de tickets, avatares) no se guardan en PostgreSQL: inflarían la base de datos, harían lentos los respaldos y no se pueden servir con URLs directas. Se usa **MinIO**, un almacenamiento de objetos compatible con la API de Amazon S3, en el contenedor `chainguard/minio` (usuario nonroot y bind mount, como en las prácticas anteriores).

Organización de llaves

```
finanzas-esp/                              (bucket, se crea al iniciar la API)
└── users/<user_id>/
    ├── avatar/avatar.jpg
    └── tickets/2026/10/<ticket_id>.jpg       (o .png / .webp / .pdf)
```

Todo lo de un usuario cuelga de su prefijo, así borrar la cuenta es un solo `delete_prefix`. Agrupar por año/mes facilita respaldos y reportes.

Medidas implementadas

- **Validación por magic bytes**: se lee la firma binaria del archivo (FF D8 FF = JPEG, 89 50 4E 47 = PNG, RIFF..WEBP, %PDF-). Un .exe renombrado a .jpg se rechaza con 415 aunque el cliente diga image/jpeg.
- **Límite de tamaño** configurable (5 MB): se leen como máximo 5 MB + 1 byte y se responde 413.
- **Duplicados**: SHA-256 del contenido + UNIQUE(user_id, image_sha256) -> 409 si se sube el mismo ticket.
- **Consistencia**: primero se sube a MinIO y luego se guarda en BD; si la BD falla, se borra el objeto (compensación) para no dejar huérfanos.
- **URLs prefirmadas** de 15 minutos: el navegador descarga directo de MinIO sin pasar por la API y sin hacer público el bucket. Se firman con `MINIO_PUBLIC_ENDPOINT` para que funcionen también desde el ESP32 en la red local.
- **Streaming**: GET /tickets/{id}/image entrega el archivo en bloques de 32 KB para clientes que no alcanzan MinIO.
- **Aislamiento**: un usuario que pide el ticket de otro recibe 404 (no se revela que existe).
- **Tolerancia a fallos**: si MinIO no responde la API sigue arriba; /health informa `storage: error` y los endpoints de archivos responden 503.



SUPERMERCADO LA ESQUINA
Av. Reforma 123, CDMX
RFC: SLE010101AB1
Fecha: 24/10/2026 18:42

LECHE ENTERA 1L	2	57.00
PAN DE CAJA	1	45.00
HUEVO 12 PZ	1	52.90
MANZANA KG	1.25	49.88
CAFE MOLIDO 400G	1	119.00
JABON LIQUIDO	1	64.50

TOTAL **388.**
EFECTIVO 500.00
CAMBIO 111.73

GRACIAS POR SU COMPRA

Figura 4. Ticket de ejemplo (samples/ticket_ejemplo.jpg) usado en las pruebas.

2.8 Integración con el ESP32

El dispositivo es un ESP32-S3 con **ESP-Claw**, el firmware de agente de Espressif. En lugar de programar en C, se le entrega un archivo `SKILL.md` que describe en lenguaje natural qué endpoints llamar con su herramienta `http_request`.

Por seguridad, el ESP32 **no usa la contraseña del usuario**. El flujo es:

1. En el panel web el usuario vincula un dispositivo y recibe una llave `fesp_...` (se muestra una sola vez; en la BD queda su SHA-256).
2. La llave se pega en el SKILL.md. El ESP32 la cambia por un JWT de tipo *device* de 60 minutos.
3. Con ese token sube tickets en JSON/base64 (POST /tickets/base64), porque http_request no puede armar multipart con bytes binarios. Los tickets quedan con source=esp32 y el device_id.
4. Si el ESP32 se pierde, el usuario lo revoca desde el panel y deja de funcionar al instante.

Limitación conocida: el base64 debe ir completo dentro del cuerpo, por lo que desde el ESP32 solo es práctico subir imágenes pequeñas (la muestra de 180×280 px pesa 6 KB). Para fotos grandes la v02 añadirá la ruta por Telegram ya probada en el proyecto MiniBot (el ESP envía la referencia y el servidor descarga la foto).

2.9 Modelos de inteligencia artificial

La IA no forma parte del alcance de la v01, pero el esquema ya está preparado para ella (`status`, `ocr_raw_text`, `ai_model`, `ai_confidence`, `ticket_items`, `categories`). Modelos seleccionados:

Modelo	Tarea	Justificación
Gemini 2.5 Flash (multimodal)	Leer la imagen del ticket y devolver JSON: tienda, fecha, total, productos, cantidades y precios.	Lee la imagen directamente (sin OCR separado), maneja tickets arrugados o inclinados, admite salida estructurada con esquema JSON y su costo por imagen es muy bajo. Ya se usó en proyectos previos del curso.
Tesseract OCR + LLM de texto (Granite / Gemini)	Respaldo cuando no hay API multimodal.	Tesseract es gratuito y local; el LLM convierte el texto ruidoso del OCR en JSON. El texto queda en ocr_raw_text para auditoría.
LLM clasificador	Asignar cada producto a una categoría.	Los tickets usan abreviaturas ("LCHE ENT 1L"); un LLM las entiende mejor que reglas por palabra clave. Se le pasa la lista de categorías de la BD para que solo elija entre ellas.
Agente LLM de ESP-Claw	Interpretar lo que el usuario pide en el chat del ESP32.	Permite "sube este ticket" o "¿cuánto gasté?" sin interfaz gráfica; el SKILL.md acota qué puede hacer.

Flujo previsto (v02): ticket *pendiente* -> *procesando* -> el modelo devuelve JSON validado con Pydantic -> se crean los `ticket_items` -> *procesado*. Si la confianza es baja o el JSON no valida, el ticket queda en *error* para captura manual. Así la IA nunca escribe datos sin validar en la base de datos.

2.10 Entorno, pruebas y evidencias

El proyecto usa un **entorno virtual** (`.venv`) con dependencias fijadas por rango en `requirements.txt` (y `requirements-dev.txt` para pruebas y para generar este documento). Los secretos viven en `.env`, excluido de git; el repositorio incluye `.env.example`. El script `iniciar.ps1` crea el venv, el .env con una SECRET_KEY nueva, levanta Docker, aplica migraciones y arranca uvicorn con `--host 0.0.0.0` para que el ESP32 alcance el servidor.

Pruebas automatizadas (pytest)

```
tests/test_auth.py::test_hash_argon2id_con_sal_aleatoria PASSED
tests/test_auth.py::test_rehash_cuando_cambian_parametros PASSED
tests/test_auth.py::test_registro_no_expone_hash PASSED
tests/test_auth.py::test_registro_duplicado_y_politica_password PASSED
tests/test_auth.py::test_login_con_usuario_o_correo PASSED
tests/test_auth.py::test_claims_del_access_token PASSED
tests/test_auth.py::test_ruta_protegida_requiere_token PASSED
tests/test_auth.py::test_refresh_no_sirve_como_access PASSED
tests/test_auth.py::test_rotacion_y_deteccion_de_reuso PASSED
tests/test_auth.py::test_logout_revoca_refresh PASSED
tests/test_auth.py::test_cambio_password_cierra_sesiones PASSED
tests/test_auth.py::test_listado_solo_admin PASSED
tests/test_storage_y_tickets.py::test_avatar_sube_a_minio_y_da_url_prefirmada PASSED
tests/test_storage_y_tickets.py::test_avatar_valida_contenido_real PASSED
tests/test_storage_y_tickets.py::test_subir_ticket_multipart PASSED
tests/test_storage_y_tickets.py::test_ticket_pdf_y_limite_de_tamano PASSED
tests/test_storage_y_tickets.py::test_ticket_duplicado PASSED
tests/test_storage_y_tickets.py::test_detalle_listado_descarga_y_borrado PASSED
tests/test_storage_y_tickets.py::test_aislamiento_entre_usuarios PASSED
tests/test_storage_y_tickets.py::test_flujo_esp32_llave_token_y_base64 PASSED
tests/test_storage_y_tickets.py::test_base64_invalido PASSED
tests/test_storage_y_tickets.py::test_borrar_cuenta_limpia_minio PASSED
============================== 22 passed in 8.15s ==============================
```

Flujo completo contra PostgreSQL y MinIO reales (scripts/demo_flujo.py)

```
[01] GET /health                                        -> HTTP 200 {'status': 'healthy',
     'database': 'ok', 'storage': 'ok'}
[02] Registro (Argon2id)                                -> HTTP 201 id=497a4660-1a2f-442f-
     8097-eae5e06d23fd
[03] Registro con contraseña débil (rechazo)            -> HTTP 422
[04] Login OAuth2 -> access + refresh                   -> HTTP 200 type=access exp_in=180
     0s
[05] Login con contraseña incorrecta                    -> HTTP 401
[06] GET /users/me sin token                            -> HTTP 401
[07] GET /users/me con Bearer                           -> HTTP 200 username=demo_2b6489
[08] PUT /users/me/avatar (MinIO)                       -> HTTP 200 has_avatar=True
[09] URL prefirmada del avatar (descarga directa)       -> HTTP 200 http://localhost:9030/
     finanzas-esp/users/497a4660-1a2f-442f-8097-eae5e06d23fd/avatar/avatar.jpg
[10] POST /tickets/ (multipart -> MinIO)                -> HTTP 201 status=pendiente sourc
     e=web
[11] Mismo ticket otra vez (SHA-256 duplicado)          -> HTTP 409
[12] Archivo falso .jpg (magic bytes)                   -> HTTP 415
[13] POST /devices/ (llave del ESP32)                   -> HTTP 201 key=fesp_KXhJjYB...
[14] POST /auth/device-token                            -> HTTP 200 type=device
[15] ESP32: POST /tickets/base64                        -> HTTP 201 source=esp32
[16] ESP32 intenta borrar la cuenta (prohibido)         -> HTTP 403
[17] GET /tickets/                                      -> HTTP 200 X-Total-Count=2
[18] GET /tickets/{id} (URL prefirmada)                 -> HTTP 200 items=[]
[19] GET /tickets/{id}/image (stream)                   -> HTTP 200 29496 bytes
```

```
[20] POST /auth/refresh (rotación)                        -> HTTP 200
[21] Reusar refresh viejo (detección de robo)             -> HTTP 401
[22] Refresh nuevo tras la alarma (revocado)              -> HTTP 401

Objetos en MinIO del usuario (3):
    users/497a4660-1a2f-442f-8097-eae5e06d23fd/avatar/avatar.jpg
    users/497a4660-1a2f-442f-8097-eae5e06d23fd/tickets/2026/10/745d7965-8460-49ab-8ffc-a03b6
      880cab1.jpg
    users/497a4660-1a2f-442f-8097-eae5e06d23fd/tickets/2026/10/a1ab5bdf-51af-43fa-8c40-69227
      672a216.jpg
```

Estado en la base de datos

Se observa que solo se guarda el hash Argon2id, que los refresh tokens usados quedaron revocados por la rotación y la detección de reutilización, que de los dispositivos solo se conserva el SHA-256 de la llave, y que los tickets apuntan a llaves de MinIO.

```
 username   |                       hashed_password                        | role
------------+--------------------------------------------------------------+------
 demo_559911 | $argon2id$v=19$m=19456,t=2,p=1$Z2wNoTTGmHOOce59z/mfsw$i2z2fU | user
 demo_2b6489 | $argon2id$v=19$m=19456,t=2,p=1$iRGC0LoXohTCOCdEaI0xBg$nEb/2H | user
(2 rows)

     jti      | revocado |          expires_at
--------------+----------+-------------------------------
 99264497b6bc | t        | 2026-11-03 03:37:11.617676+00
 11c83268dca8 | t        | 2026-11-03 03:37:13.771839+00
 3bba2ec54e1c | t        | 2026-11-03 03:37:37.48103+00
 b5e56d4bf789 | t        | 2026-11-03 03:37:39.347887+00
(4 rows)

     name      |  key_prefix  |       key_hash       | is_active
---------------+--------------+----------------------+-----------
 ESP32-S3 demo | fesp_kld6ekV | f1f06f40818926017b6f | t
 ESP32-S3 demo | fesp_KXhJjYB | 47e94f8001f33134c81b | t
(2 rows)

 source |  status   | image_size |                            left

--------+-----------+------------+-------------------------------------------------------------
      ----
 web    | pendiente |      29496 | users/9342f8b7-3eda-4b8c-938b-1e22b4c91022/tickets/2026/1
      0/5
 esp32  | pendiente |       6346 | users/9342f8b7-3eda-4b8c-938b-1e22b4c91022/tickets/2026/1
      0/9
 web    | pendiente |      29496 | users/497a4660-1a2f-442f-8097-eae5e06d23fd/tickets/2026/1
      0/a
 esp32  | pendiente |       6346 | users/497a4660-1a2f-442f-8097-eae5e06d23fd/tickets/2026/1
      0/7
(4 rows)
```

2.11 Repositorio y rama v01

El código está en GitHub. El trabajo de este entregable vive en la rama **v01**; cada entregable posterior tendrá su propia rama (v02, v03) para poder revisar la evolución del proyecto.

```
git init
git checkout -b v01
git add .
git commit -m "v01: modelo User, hasheo Argon2id, tokens JWT y MinIO"
git remote add origin https://github.com/OwenPMorales/FinanzasESP.git
git push -u origin v01
```

Quedan fuera del repositorio (.gitignore): `.venv/`, `.env`, `minio_data/`, bases SQLite y cachés de Python.

3. Conclusión

La v01 de FinanzasESP deja lista la base sobre la que se construirá el control de finanzas: una identidad de usuario sólida, contraseñas protegidas con el algoritmo recomendado actualmente, un sistema de tokens que distingue entre personas y dispositivos, y un almacenamiento de archivos escalable e independiente de la base de datos.

Las decisiones se tomaron pensando en riesgos reales y no solo en que "funcione": Argon2id en lugar de bcrypt o SHA-256 por su resistencia a GPU; refresh tokens con rotación y detección de reutilización para limitar el daño de un token robado; llaves propias y permisos reducidos para el ESP32 en lugar de compartir la contraseña; validación del contenido real de los archivos y URLs prefirmadas en lugar de un bucket público. Todo está respaldado por 22 pruebas automatizadas y por una demostración completa contra PostgreSQL y MinIO en Docker.

El modelo de datos ya contempla productos, categorías y la trazabilidad de la IA, por lo que la v02 puede concentrarse en integrar Gemini para leer los tickets sin modificar lo construido. Con eso, la v03 podrá ofrecer el objetivo final: ver en qué se gasta, cómo cambian los precios y si se respeta el presupuesto, consultable incluso desde el ESP32.

Referencias

- OWASP. *Password Storage Cheat Sheet*. cheatsheetseries.owasp.org
- Biryukov, A., Dinu, D., Khovratovich, D. (2016). *Argon2: the memory-hard function for password hashing*. RFC 9106.
- Jones, M., Bradley, J., Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519.
- OWASP. *JSON Web Token Cheat Sheet* y *OAuth 2.0 Security Best Current Practice* (rotación de refresh tokens).
- FastAPI. *Security — OAuth2 with Password (and hashing), Bearer with JWT tokens*. fastapi.tiangolo.com
- SQLModel. *Documentation*. sqlmodel.tiangolo.com

- MinIO. *Python Client API Reference* y *Presigned URLs*. min.io/docs
- Espressif. *ESP-Claw*: capacidades cap_http_request y skills. github.com/espressif/esp-claw
- Google. *Gemini API — Image understanding and structured output*. ai.google.dev